

Grado en Ingeniería del Software
Doble Grado en Matemática Computacional e Ingeniería del Software
Doble Grado en Física Computacional e Ingeniería del Software



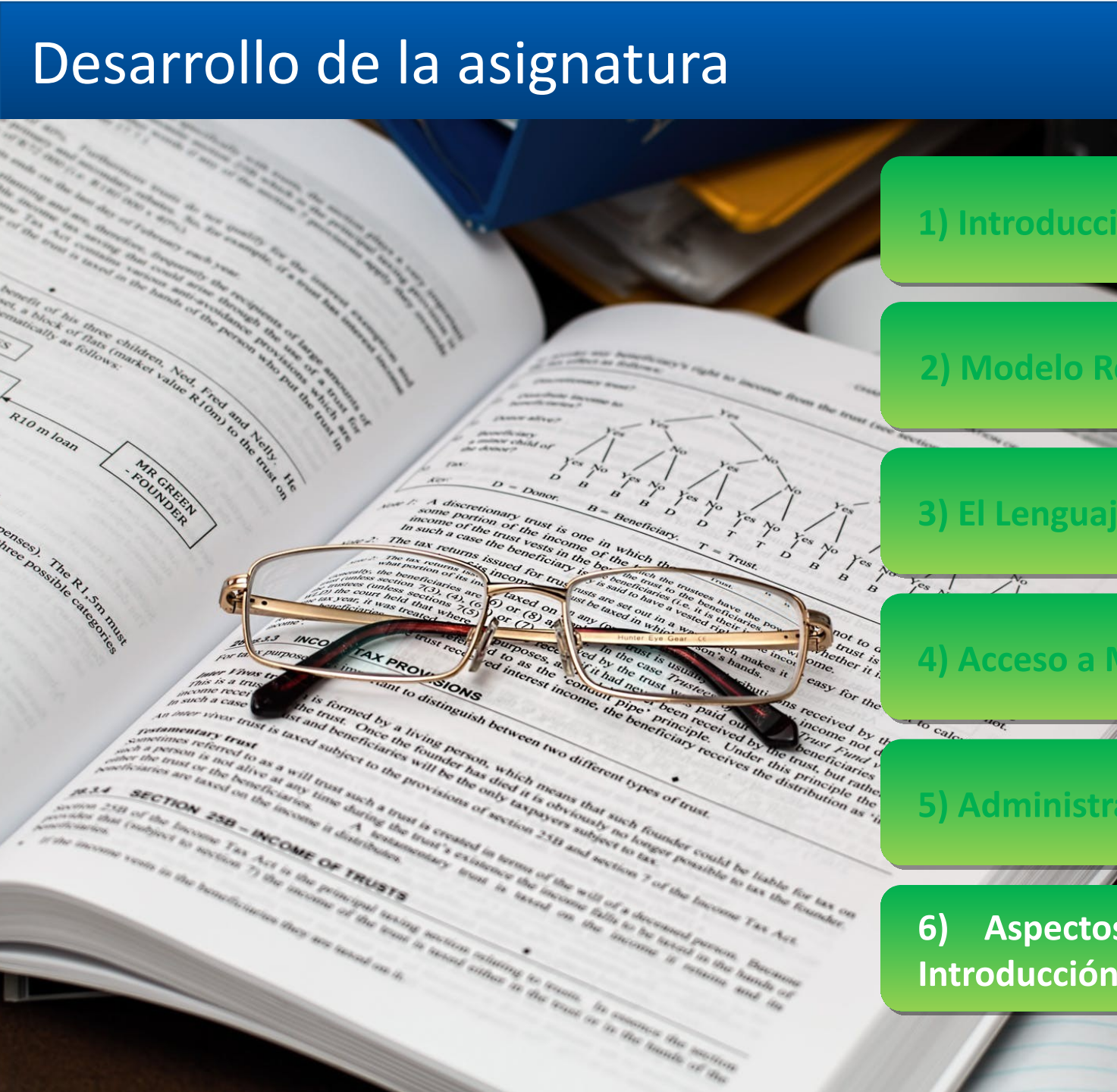
Bases de Datos

Tema 6

Rafael Socas Gutiérrez
Amador Maho Etohá
Ana Isabel Sánchez González

Enero 2025





1) Introducción a las Bases de Datos

2) Modelo Relacional y Algebra Relacional

3) El Lenguaje SQL y SQL Avanzado

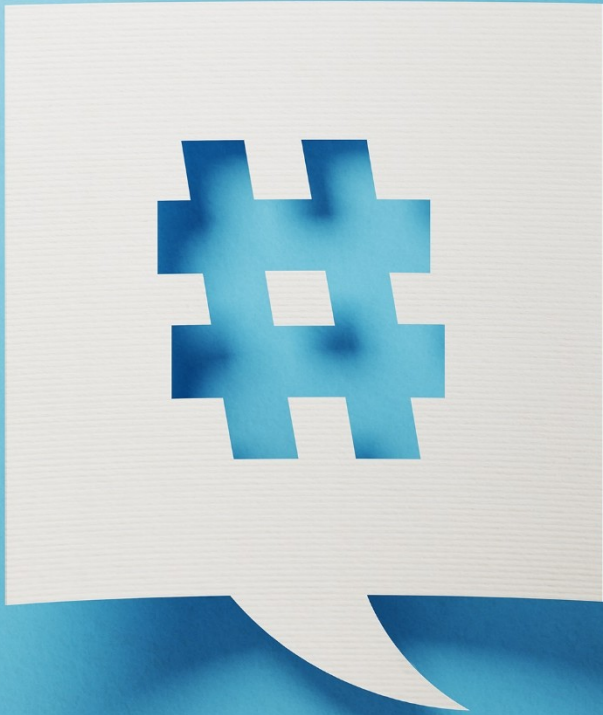
4) Acceso a MySQL, Almacenamiento e Indexado

5) Administración de Bases de Datos en MySQL

6) Aspectos Avanzados sobre BBDD Relacionales e Introducción a las BBDD No Relacionales: MongoDB

6

Aspectos Avanzados sobre BBDD Relacionales e Introducción a Bases de Datos No Relacionales: MongoDB



1. *Lenguajes Procedurales para BBDD*
2. *Procedimientos Almacenados*
3. *SQL Injection*
4. *Conceptos Generales BBDD No Relacionales*
5. *Clasificación Bases de Datos NoSQL*
6. *MongoDB: Características Principales*
7. *Instalación MongoDB*
8. *Elementos de MongoDB*
9. *Operaciones con Documentos*
10. *Índices sobre MongoDB*

Diferencia entre lenguaje Procedural y No Procedural

Lenguaje Procedural

El código del programa se escribe como una secuencia de instrucciones. El usuario tiene que especificar "qué hacer" y también "cómo hacerlo" (procedimiento paso a paso). Estas instrucciones se ejecutan en orden secuencial.

C and C++, Java, Pascal, BASIC

Lenguaje No Procedural

El usuario sólo tiene que especificar "qué hacer" y no "cómo hacerlo". También se conoce como lenguaje aplicativo o funcional. Implica el desarrollo de las funciones a partir de otras funciones para construir funciones más complejas.

LISP, SQL, PROLOG

Factor	Lenguaje Procedural	Lenguaje No Procedural
Program code	El código del programa se escribe en forma de una secuencia de instrucciones.	Se trata de desarrollar funciones basadas en otras funciones, para construir otras funcionalidades complicadas.
Semántica y Complejidad	Su semántica es compleja.	Su semántica es sencilla en comparación con los lenguajes procedurales.
Tamaño del programa	El tamaño del programa suele ser grande.	Los programas son de pequeño tamaño.
Eficiencia	La eficiencia global es alta.	La eficiencia global del lenguaje no procedural es baja en comparación con el procedural.
Tiempo Real	No es adecuado para aplicaciones en las que el tiempo es un factor crítico.	Es ideal para aplicaciones en las que el tiempo es un factor crítico.

2. Procedimientos Almacenados

Procedimientos, Funciones y Triggers en MySQL

Los procedimientos, funciones y *triggers* de MySQL, que son objetos que contienen código SQL y se almacenan asociados a una base de datos.

- **Procedimiento almacenado:** se crea con la sentencia `CREATE PROCEDURE` y se invoca con la sentencia `CALL`. Un procedimiento puede tener cero o muchos parámetros de entrada y cero o muchos parámetros de salida.
- **Función almacenada:** se crea con la sentencia `CREATE FUNCTION` y se invoca con la sentencia `SELECT` o dentro de una expresión. Una función puede tener cero o muchos parámetros de entrada y siempre devuelve un valor, asociado al nombre de la función.
- **Trigger:** se usa la sentencia `CREATE TRIGGER` y tiene que estar asociado a una tabla. Un trigger se activa cuando ocurre un evento de inserción, actualización o borrado, sobre la tabla a la que está asociado.



Procedimientos vs. Funciones

- *La primera y más obvia diferencia, es que las funciones siempre devuelven un valor, mientras que un procedimiento almacenado puede que retorne un valor o puede que no lo haga. Es decir que puede comportarse como un método o como una función, haciendo una analogía con el desarrollo de software.*
- *Otra diferencia muy importante para los desarrolladores es que los procedimientos almacenados pueden ser invocados desde el entorno de desarrollo, es decir desde .NET, pero las funciones no. Así que, al momento de desarrollar, siempre nos comunicaremos con procedimientos almacenados.*
- *Las funciones de hecho no pueden ser invocadas por sí solas, mientras que los procedimientos almacenados sí.*



2. Procedimientos Almacenados

Ejemplo de Procedimiento (1/2)

Enunciado

Escriba un procedimiento llamado **listar_productos** que reciba como entrada el nombre de la gama y muestre un listado de todos los productos que existen dentro de esa gama. Este procedimiento no devuelve ningún parámetro de salida, lo que hace es mostrar el listado de los productos.

Definición

```
DELIMITER $$  
DROP PROCEDURE IF EXISTS listar_productos$$  
CREATE PROCEDURE listar_productos(IN gama VARCHAR(50))  
BEGIN  
    SELECT *  
    FROM producto  
    WHERE producto.gama = gama;  
END  
$$
```

Invocación

```
DELIMITER ;  
CALL listar_productos('Herramientas');  
SELECT * FROM producto;
```

Ver los Procedimientos

```
SHOW CREATE PROCEDURE listar_productos;  
SHOW PROCEDURE STATUS WHERE Db='database';
```

2. Procedimientos Almacenados

Ejemplo de Procedimiento (2/2)

Enunciado

Escriba un procedimiento llamado **contar_productos** que reciba como entrada el nombre de la gama y devuelva el número de productos que existen dentro de esa gama.

Solución 1

```
DELIMITER $$
DROP PROCEDURE IF EXISTS contar_productos$$
CREATE PROCEDURE contar_productos(IN gama VARCHAR(50), OUT total INT UNSIGNED)
BEGIN
    SET total = (
        SELECT COUNT(*)
        FROM producto
        WHERE producto.gama = gama);
END
$$

DELIMITER ;
CALL contar_productos('Herramientas', @total);
SELECT @total;
```

Solución 2

```
DELIMITER $$
DROP PROCEDURE IF EXISTS contar_productos$$
CREATE PROCEDURE contar_productos(IN gama VARCHAR(50), OUT total INT UNSIGNED)
BEGIN
    SELECT COUNT(*)
    INTO total
    FROM producto
    WHERE producto.gama = gama;
END
$$

DELIMITER ;
CALL contar_productos('Herramientas', @total);
SELECT @total;
```

Ejemplo de Función

Enunciado

Escriba una función llamada **contar_productos** que reciba como entrada el nombre de la gama y devuelva el número de productos que existen dentro de esa gama.

Ver las Funciones

```
SHOW CREATE FUNCTION contar_productos;  
SHOW FUNCTION STATUS WHERE Db='database';
```

Solución

```
DELIMITER $$  
DROP FUNCTION IF EXISTS contar_productos$$  
CREATE FUNCTION contar_productos(gama VARCHAR(50))  
    RETURNS INT UNSIGNED DETERMINISTIC  
BEGIN  
    -- Paso 1. Declaramos una variable local  
    DECLARE total INT UNSIGNED;  
  
    -- Paso 2. Contamos los productos  
    SET total = (  
        SELECT COUNT(*)  
        FROM producto  
        WHERE producto.gama = gama);  
  
    -- Paso 3. Devolvemos el resultado  
    RETURN total;  
END  
$$  
  
DELIMITER ;  
SELECT contar_productos('Herramientas');
```

Enunciado

Ejemplo de Trigger (1/2)

Crea una base de datos llamada test que contenga una tabla llamada alumnos con las siguientes columnas.

Tabla alumnos:

*id (entero sin signo)
nombre (cadena de caracteres)
apellido1 (cadena de caracteres)
apellido2 (cadena de caracteres)
nota (número real)*

Una vez creada la tabla escriba dos triggers con las siguientes características:

Trigger 1: *trigger_check_nota_before_insert*

Se ejecuta sobre la tabla alumnos.

Se ejecuta antes de una operación de inserción.

Si el nuevo valor de la nota que se quiere insertar es negativo, se guarda como 0.

Si el nuevo valor de la nota que se quiere insertar es mayor que 10, se guarda como 10.

Trigger2 : *trigger_check_nota_before_update*

Se ejecuta sobre la tabla alumnos.

Se ejecuta antes de una operación de actualización.

Si el nuevo valor de la nota que se quiere actualizar es negativo, se guarda como 0.

Si el nuevo valor de la nota que se quiere actualizar es mayor que 10, se guarda como 10.

Una vez creados los triggers escriba varias sentencias de inserción y actualización sobre la tabla alumnos y verifica que los triggers se están ejecutando correctamente.

Solución

Ejemplo de Trigger (2/2)

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  apellido1 VARCHAR(50) NOT NULL,
  apellido2 VARCHAR(50),
  nota FLOAT
);
```

```
-- Paso 3
DELIMITER $$
DROP TRIGGER IF EXISTS trigger_check_nota_before_insert$$
CREATE TRIGGER trigger_check_nota_before_insert
BEFORE INSERT
ON alumnos FOR EACH ROW
BEGIN
  IF NEW.nota < 0 THEN
    set NEW.nota = 0;
  ELSEIF NEW.nota > 10 THEN
    set NEW.nota = 10;
  END IF;
END

DELIMITER $$
DROP TRIGGER IF EXISTS trigger_check_nota_before_update$$
CREATE TRIGGER trigger_check_nota_before_update
BEFORE UPDATE
ON alumnos FOR EACH ROW
BEGIN
  IF NEW.nota < 0 THEN
    set NEW.nota = 0;
  ELSEIF NEW.nota > 10 THEN
    set NEW.nota = 10;
  END IF;
END
```

```
-- Paso 4
DELIMITER ;
INSERT INTO alumnos VALUES (1, 'Pepe', 'López', 'López', -1);
INSERT INTO alumnos VALUES (2, 'María', 'Sánchez', 'Sánchez', 11);
INSERT INTO alumnos VALUES (3, 'Juan', 'Pérez', 'Pérez', 8.5);

-- Paso 5
SELECT * FROM alumnos;

-- Paso 6
UPDATE alumnos SET nota = -4 WHERE id = 3;
UPDATE alumnos SET nota = 14 WHERE id = 3;
UPDATE alumnos SET nota = 9.5 WHERE id = 3;

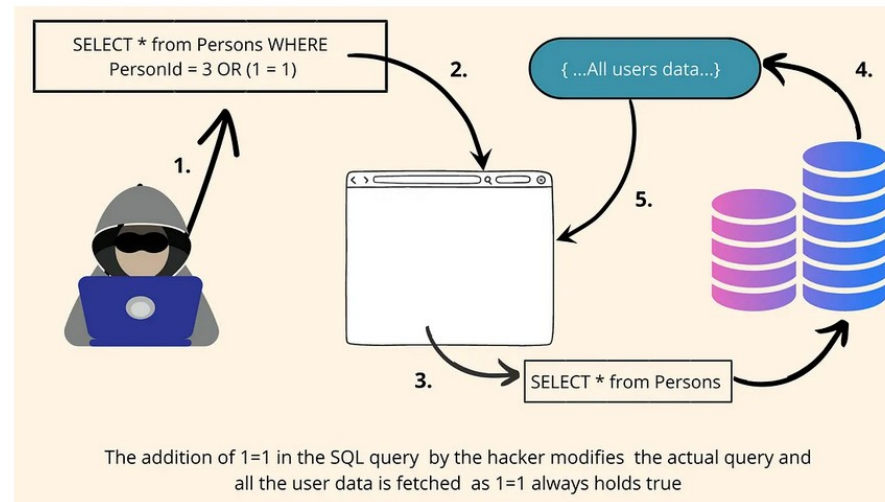
-- Paso 7
SELECT * FROM alumnos;
```

Ver los Triggers

```
SHOW CREATE TRIGGER trigger_check_nota_before_insert;
SHOW CREATE TRIGGER trigger_check_nota_before_update;
SHOW TRIGGERS FROM test;
```

¿Qué es la inyección SQL?

Una inyección SQL (del inglés SQL injection) se define como la explotación de una vulnerabilidad en los sistemas de bases relacionales accediendo a sus datos por medio del **lenguaje SQL**. El atacante se aprovecha de aquellos fallos de seguridad en la superficie de la base de datos que no han sido correctamente enmascarados y que contienen metacaracteres como el guion doble, las comillas o el punto y coma. Estos caracteres representan funciones especiales para el intérprete de SQL y permiten la influencia externa sobre las instrucciones ejecutadas. Es común que las inyecciones SQL se realicen en conjunto con programas PGP y ASP, que **dependen de interfaces más antiguas**. Allí, algunas de las entradas no cuentan con el correcto enmascaramiento, convirtiéndose así en el blanco perfecto para estos ataques.



Analizar el siguiente artículo para entender cómo puede afectar esta vulnerabilidad

<https://www.ionos.es/digitalguide/servidores/seguridad/inyeccion-sql-principios-y-precauciones/>

Fuente: <https://betterprogramming.pub/stop-doing-this-in-your-sql-queries-if-you-already-are-252b3f8c1029>

4. Conceptos Generales BBDD No Relacionales

Partimos de las BBDD Relacionales

Usos

Desde los años 80, la mayoría de los sistemas de información almacenan sus datos en gestores relacionales:

- Aplicaciones de banca
- CRM (Customer Relationship Management)
- ERP (Enterprise Resource Planning)
- Sistemas sanitarios
- Sistemas académicos
- Etc.

Básicamente, estos recogen procesos operacionales que gestionan datos simples y estructurados (p.ej. transacciones bancarias, compras,...)

Características

Las BD relacionales (R-DBMS) se caracterizan por:

- Utilizar un modelo de datos simple (basado en tablas y relaciones entre tablas)
- Ofrecer herramientas para garantizar la integridad de datos y la consistencia de la información (ACID)
- Utilizar un lenguaje de interrogación estándar, simple y potente
- Proporcionar utilidades para asegurar el acceso, manipulación y la privacidad de los datos
- Ofrecer utilidades para la auditoría y recuperación de datos
- Garantizar la independencia del esquema lógico y físico

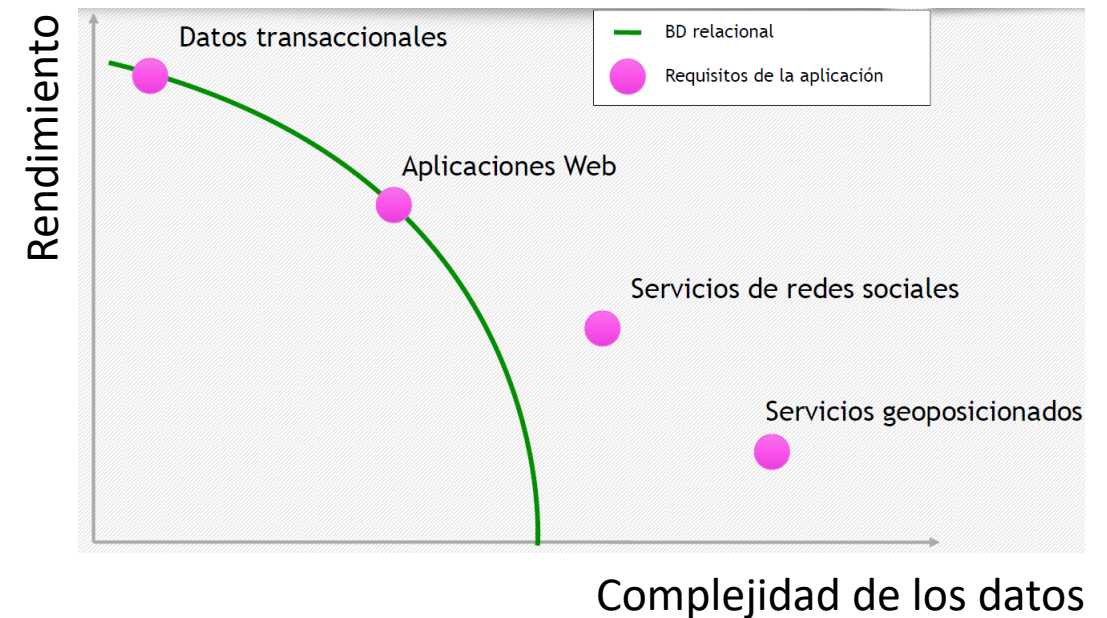
4. Conceptos Generales BBDD No Relacionales

Aparece el Término Big Data

Características del Big Data

- **Volume:** gran cantidad de datos
- **Velocity:** necesidad de ser analizados rápidamente
- **Variety:** datos estructurados y no estructurados, densos y dispersos, conectados y desconectados
- **Variability:** datos cuyo significado cambia constantemente
- **Veracity:** veracidad, precisión
- **Visualisation:** presentar los datos de forma comprensible
- **Value:** extraer información para la toma de decisiones

Impacto en el Rendimiento



4. Conceptos Generales BBDD No Relacionales

Solución: Bases de Datos NoSQL

Principios

NoSQL se basa en 4 principios:

- El control transaccional ACID no es importante. Implementa BASE.
- Los *JOINS* tampoco lo son. En especial los complejos y distribuidos. Se persigue la desnormalización.
- Algunos elementos relacionales son necesarios y aconsejables: claves (**keys**).
- Gran capacidad de escalabilidad y de replicación en múltiples servidores.

Características

- *Basically Available*, está operativo la mayoría del tiempo
- *Soft state*, los datos en las diferentes réplicas no tienen que ser mutuamente consistentes en todo momento.
- *Eventually Consistent*, se asegura la consistencia solo después de que pase cierto tiempo.

En resumen:

- Consistencia débil – datos obsoletos *OK*
- Prima la disponibilidad
- Respuestas aproximadas *OK*
- Agresivamente optimista, disponibilidad, aunque fallen nodos

4. Conceptos Generales BBDD No Relacionales

Ventajas vs Inconvenientes NoSQL

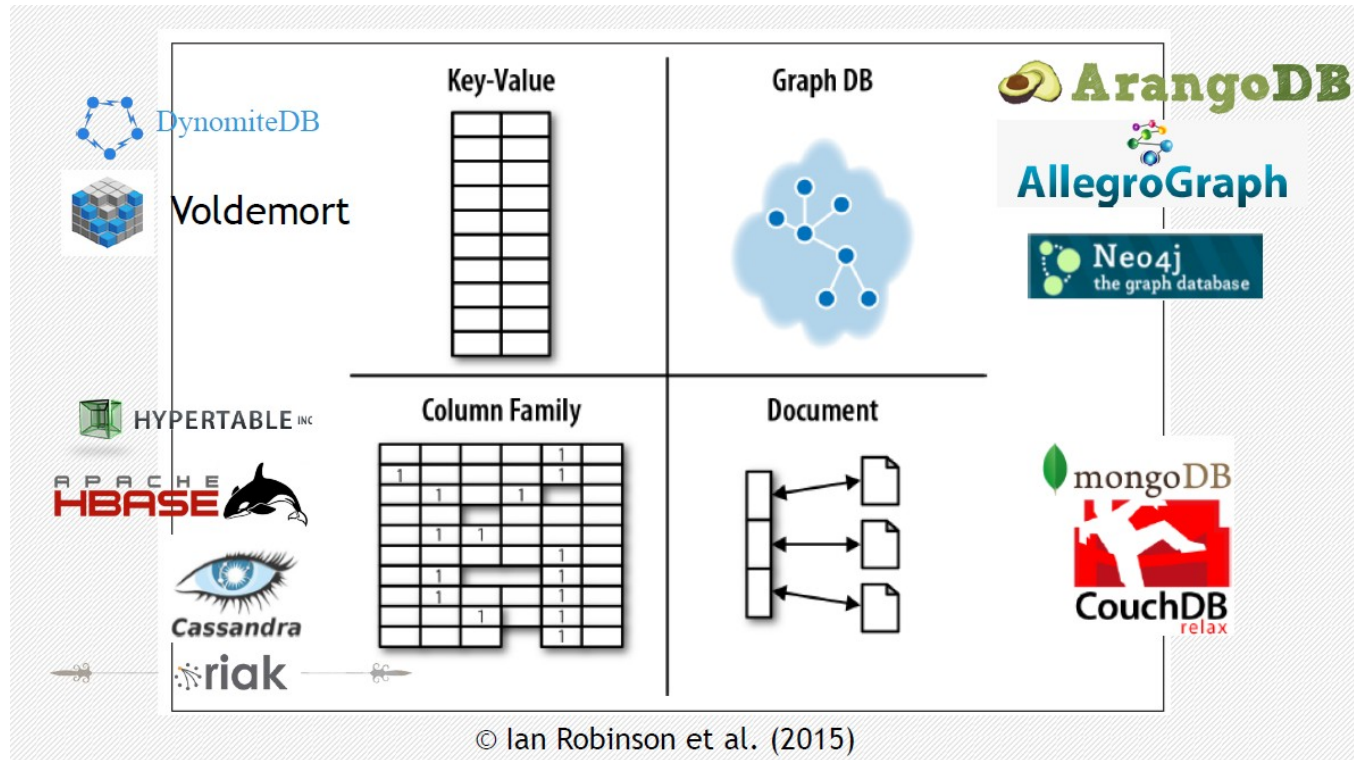
Ventajas

- Masivamente escalables
- Alta disponibilidad
- Bajo coste comparado con soluciones similares de la misma escala
- Generalmente, elasticidad predecible
- Esquema flexible, adecuado para datos dispersos y semiestructurados

Inconvenientes

- Capacidad de interrogación limitada
- Consistencia eventual à su programación no es intuitiva
- No estandarizado à no portabilidad
- El desarrollo de aplicaciones clientes más complejo
- Carecen de herramientas de control de acceso (seguridad)

5. Clasificación Bases de Datos NoSQL



Aplicaciones

Key-Value:

- Información de sesión en aplicaciones web
- Perfiles y preferencias de usuario
- Carritos de la compra

Document:

- Blogs
- Event logging
- Datos operacionales y meta datos para aplicaciones web y móviles

Column Family:

- Blogs
- Event logging
- Contadores

Graph DB:

- Almacenar datos conectados
- Aplicaciones espaciales o de rutas
- Motores de recomendación

¿Qué es MongoDB? MongoDB es una base de datos de documentos que ofrece una gran escalabilidad y flexibilidad, y un modelo de consultas e indexación avanzado.

Características

- MongoDB **almacena datos en documentos flexibles similares a JSON**, por lo que los campos pueden variar entre documentos y la estructura de datos puede cambiarse con el tiempo
- El modelo de documento **se asigna a los objetos en el código de su aplicación** para facilitar el trabajo con los datos
- **Las consultas ad hoc, la indexación y la agregación en tiempo real** ofrecen maneras potentes de acceder a los datos y analizarlos
- MongoDB es una **base de datos distribuida en su núcleo**, por lo que la alta disponibilidad, la escalabilidad horizontal y la distribución geográfica están integradas y son fáciles de usar
- MongoDB es **de uso gratuito**. Las versiones lanzadas antes del 16 de octubre de 2018 se publican bajo licencia AGPL. Todas las versiones posteriores al 16 de octubre de 2018, incluidos los parches lanzados para versiones anteriores, se publican bajo [Licencia pública del lado del servidor \(SSPL\) v1](https://www.mongodb.com/es/what-is-mongodb).



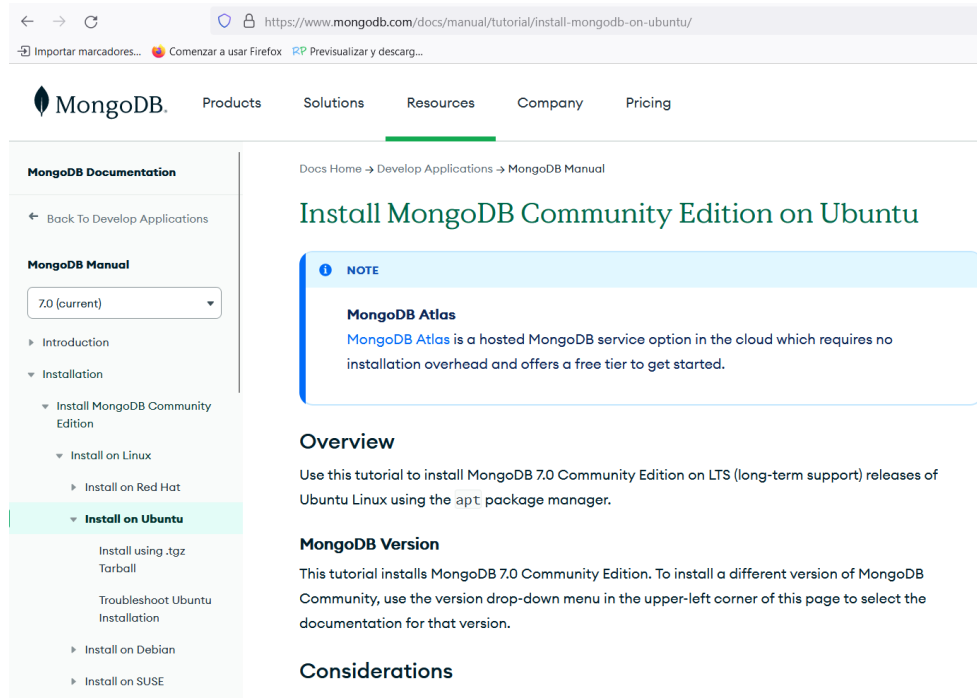
```
1 {  
2   _id: "5cf0029caff5056591b0ce7d",  
3   firstname: 'Jane',  
4   lastname: 'Wu',  
5   address: {  
6     street: '1 Circle Rd',  
7     city: 'Los Angeles',  
8     state: 'CA',  
9     zip: '90404'  
10  }  
11 }
```

Para instalar un **servidor de MongoDB** basta con visitar (www.mongodb.com) y seguir el tutorial correspondiente según el Sistema Operativo que tengamos.

Instalación MongoDB en Ubuntu

Tutorial para Ubuntu

<https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-ubuntu/>



A fecha de hoy (Oct/2023) las versiones compatibles más estables son:



20.04 (Focal)



Primeros Pasos

Arranque del Servidor

service mongod start arranca el servicio
service mongod status comprueba el estado

```
root@MySQLSERVER: /home/rsocas
root@MySQLSERVER:/home/rsocas# service mongod start
root@MySQLSERVER:/home/rsocas# service mongod status
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; disabled; vendor prese
   Active: active (running) since Thu 2023-10-19 12:01:54 CEST; 3s ago
     Docs: https://docs.mongodb.org/manual
    Main PID: 3298 (mongod)
      Memory: 211.4M
        CGroup: /system.slice/mongod.service
                └─3298 /usr/bin/mongod --config /etc/mongod.conf

oct 19 12:01:54 MySQLSERVER systemd[1]: Started MongoDB Database Server.
oct 19 12:01:54 MySQLSERVER mongod[3298]: {"t":{"$date":"2023-10-19T10:01:54.66"}
lines 1-11/11 (END)
```

Acceso al Servicio

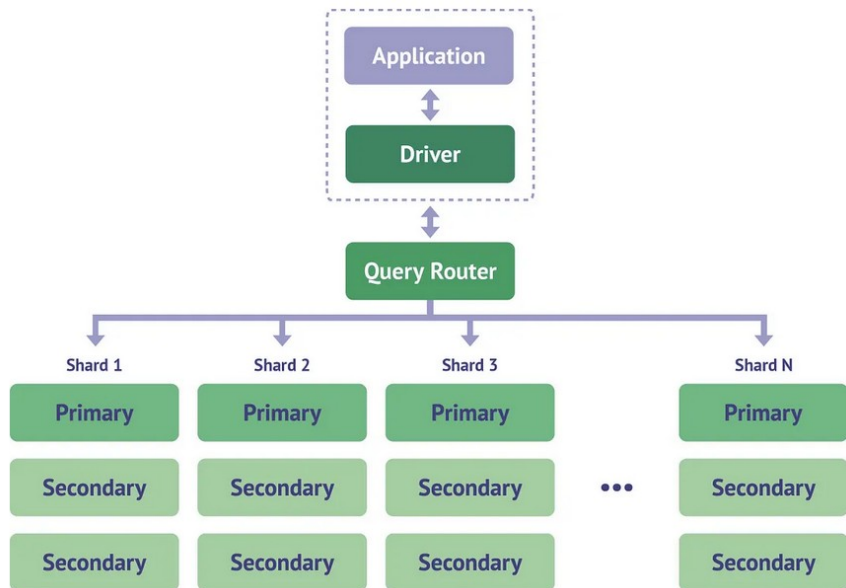
mongo acceso al MongoDB
show dbs muestra las bases de datos
help muestra la ayuda

```
root@MySQLSERVER: /home/rsocas
> show dbs
admin    0.000GB
config  0.000GB
local    0.000GB
> help
db.help()                help on db methods
db.mycoll.help()          help on collection methods
sh.help()                 sharding helpers
rs.help()                 replica set helpers
help admin                administrative help
help connect              connecting to a db help
help keys                 key shortcuts
help misc                 misc things to know
help mr                   mapreduce

show dbs                  show database names
show collections           show collections in current database
show users                show users in current database
show profile              show most recent system.profile entries with time >= 1ms
show logs                 show the accessible logger names
show log [name]           prints out the last segment of log in memory, 'global' is default
use <db_name>             set current database
db.mycoll.find()           list objects in collection mycoll
db.mycoll.find( { a : 1 } ) list objects in mycoll where a == 1
it                         result of the last line evaluated; use to further iterate
DBQuery.shellBatchSize = x set default number of items to display on shell
exit                      quit the mongo shell
```

8. Elementos de MongoDB

Arquitectura



Componentes

- **Collection:** es un grupo de Documentos MongoDB, lo que equivale a una tabla RDBMS.
- **Document:** es un conjunto de pares clave-valor, donde un valor puede ser cualquiera de los tipos de datos BSON, incluyendo arrays y otros documentos.
- **Database:** es un contenedor de colecciones que contiene todos los datos de una aplicación concreta.
- **Shard:** es una partición horizontal de datos que se reparte entre varios servidores. MongoDB utiliza sharding para distribuir los datos entre múltiples nodos y soportar la escalabilidad horizontal.
- **Replica Set:** es un grupo de servidores que mantienen copias idénticas de los mismos datos. La replicación proporciona alta disponibilidad, durabilidad y redundancia de datos.
- **Query Router:** son los componentes de un clúster MongoDB que reciben las peticiones de los clientes, determinan la ubicación de los datos requeridos y dirigen la petición al fragmento apropiado.
- **Configuration Servers:** almacenan metadatos y ajustes de configuración para un cluster MongoDB. Son responsables de mantener la asignación de shards a conjuntos de réplicas.

9. Operaciones con Documentos



Estructura Base de Datos

Base de Datos	Tienda	
Colecciones	Usuarios	Pedidos
Documentos	{Nombre: 'Juan', edad: 25}	{...}
	{Nombre: 'Felipe'}	{...}

Estructura Documentos

JSON (BSON)

```
{
  "nombre": "Juan",
  "edad": 25
  "dirección":
    {
      "ciudad": "Barcelona"
    },
  "aficiones": [
    {"nombre": "Fútbol" },
    {"nombre": "Esquí" }
  ]
}
```

9. Operaciones con Documentos

Crear una Base de Datos e Incluir Documentos

Crear Base de Base de Datos

> use blog

Crear la Colección

> db.createCollection("posts")

Incluir Documentos

```
> db.posts.insertOne({  
  title: "Post Title 1",  
  body: "Body of post.",  
  category: "News",  
  likes: 1,  
  tags: ["news", "events"],  
  date: Date()  
})
```

```
> db.posts.insertMany([  
  {  
    title: "Post Title 2",  
    body: "Body of post.",  
    category: "Event",  
    likes: 2,  
    tags: ["news", "events"],  
    date: Date()  
  },  
  {  
    title: "Post Title 3",  
    body: "Body of post.",  
    category: "Technology",  
    likes: 3,  
    tags: ["news", "events"],  
    date: Date()  
  },  
  {  
    title: "Post Title 4",  
    body: "Body of post.",  
    category: "Event",  
    likes: 4,  
    tags: ["news", "events"],  
    date: Date()  
  }  
])
```

9. Operaciones con Documentos

Acceso a los Documentos

Todos los de una Colección

>db.posts.find()

```
root@MySQLSERVER: /home/rsocas
> db.posts.find()
{ "_id" : ObjectId("653105fd8a4353e8b4e7d7"), "title" : "Post Title 1",
  "body" : "Body of post.", "category" : "News", "likes" : 1, "tags" : [
    "news", "events" ], "date" : "Thu Oct 19 2023 12:33:33 GMT+0200 (CEST)"
}
{ "_id" : ObjectId("65310613fd8a4353e8b4e7d8"), "title" : "Post Title 2",
  "body" : "Body of post.", "category" : "Event", "likes" : 2, "tags" : [
    "news", "events" ], "date" : "Thu Oct 19 2023 12:33:55 GMT+0200 (CEST)"
}
{ "_id" : ObjectId("65310613fd8a4353e8b4e7d9"), "title" : "Post Title 3",
  "body" : "Body of post.", "category" : "Technology", "likes" : 3, "tags" : [
    "news", "events" ], "date" : "Thu Oct 19 2023 12:33:55 GMT+0200 (CEST)"
}
{ "_id" : ObjectId("65310613fd8a4353e8b4e7da"), "title" : "Post Title 4",
  "body" : "Body of post.", "category" : "Event", "likes" : 4, "tags" : [
    "news", "events" ], "date" : "Thu Oct 19 2023 12:33:55 GMT+0200 (CEST)"
}
```

Sólo un Documento

>db.posts.findOne()

```
root@MySQLSERVER: /home/rsocas
> db.posts.findOne()
{
  "_id" : ObjectId("653105fd8a4353e8b4e7d7"),
  "title" : "Post Title 1",
  "body" : "Body of post.",
  "category" : "News",
  "likes" : 1,
  "tags" : [
    "news",
    "events"
  ],
  "date" : "Thu Oct 19 2023 12:33:33 GMT+0200 (CEST)"
}
```

9. Operaciones con Documentos

Querying Data

Para consultar, o filtrar, datos podemos incluir una consulta en nuestros métodos **find()** o **findOne()**. Ambos métodos de búsqueda aceptan un segundo parámetro llamado **projection**. Este parámetro es un objeto que describe qué campos incluir en los resultados.

Ejemplos de Consultas

```
>db.posts.find( {category: "News"} )
```

Muestra los documentos de la categoría News.

```
>db.posts.find({}, {title: 1, date: 1})
```

Sólo mostrará los campos de title y date en los resultados.

```
>db.posts.find({}, {_id: 0, title: 1, date: 1})
```

Se excluye el campo _id.

```
>db.posts.find({}, {category: 0})
```

Se excluye campo de categoría. Todos los demás campos se incluyen.

```
>db.posts.find({}, {title: 1, date: 0})
```

Error si intentamos especificar tanto 0 como 1 en el mismo objeto.

9. Operaciones con Documentos

Operadores

Logical

The following operators can logically compare multiple queries.

- `$and` : Returns documents where both queries match
- `$or` : Returns documents where either query matches
- `$nor` : Returns documents where both queries fail to match
- `$not` : Returns documents where the query does not match

Evaluation

The following operators assist in evaluating documents.

- `$regex` : Allows the use of regular expressions when evaluating field values
- `$text` : Performs a text search
- `$where` : Uses a JavaScript expression to match documents

Comparison

The following operators can be used in queries to compare values:

- `$eq` : Values are equal
- `$ne` : Values are not equal
- `$gt` : Value is greater than another value
- `$gte` : Value is greater than or equal to another value
- `$lt` : Value is less than another value
- `$lte` : Value is less than or equal to another value
- `$in` : Value is matched within an array

Ejemplo

Documentos que pertencencan a la categoría “Event” Y tengan menos de 3 “likes”

```
db.posts.find( { $and: [ { category: "Event" }, { likes: { $lt: 3 } } ] } )
```

```
root@MySQLSERVER: /home/rsocas
> db.posts.find( { $and: [ { category: "Event" }, { likes: { $lt: 3 } } ] } )
{ "_id" : ObjectId("65310613fd8a4353e8b4e7d8"), "title" : "Post Title 2", "body" : "Body of post.", "category" : "Event", "likes" : 2, "tags" : [ "news", "events" ], "date" : "Thu Oct 19 2023 12:33:55 GMT+0200 (CEST)" }
>
```

Principio de Funcionamiento de los Índices

Los índices en **MongoDB** se generan en forma de Árbol-B o B-Tree. Es decir, que los datos se guardan en forma de árbol, pero manteniendo los nodos balanceados. Esto incrementa la velocidad a la hora de buscar y también a la hora de devolver resultados ya ordenados. De hecho, **MongoDB** es capaz de recorrer los índices en ambos sentidos, por lo que, con un solo índice, podemos conseguir ordenación tanto ascendente como descendente.

Principales comandos para trabajar con Índices

<code>>db.posts.find({category: "News"}).explain('executionStats')</code>	<i>Muestra las estadísticas de una consulta (documentos recorridos)</i>
<code>>db.posts.createIndex({ category: 1 })</code>	<i>Crea un índice, (ejemplo campo category)</i>
<code>>db.posts.getIndexes()</code>	<i>Muestra los índices que tiene definidos una colección</i>
<code>>db.posts.dropIndexes()</code>	<i>Borra todos los índices de una colección</i>
<code>>db.posts.dropIndex("category_1")</code>	<i>Borra un índice específico de una colección (ejemplo campo category)</i>

Ejemplo de Uso de los índices

Busca por el campo Category sin Indexar

```
>db.posts.find( {category: "News"} ).explain('executionStats')
```

```
"executionStats" : {  
  "executionSuccess" : true,  
  "nReturned" : 1,  
  "executionTimeMillis" : 0,  
  "totalKeysExamined" : 0,  
  "totalDocsExamined" : 4,  
  "executionStages" : {
```

Recorre TODOS los documentos

Busca por el campo Category Indexado

```
>db.posts.createIndex({ category: 1 })  
>db.posts.find( {category: "News"} ).explain('executionStats')
```

```
"executionStats" : {  
  "executionSuccess" : true,  
  "nReturned" : 1,  
  "executionTimeMillis" : 0,  
  "totalKeysExamined" : 1,  
  "totalDocsExamined" : 1,  
  "executionStages" : {
```

Gracias a B-Tree solo 1



 Calle Playa de Liencres, 2 bis
(entrada por calle Rozabella)
Parque Europa Empresarial
Edificio Madrid
28290 Las Rozas, Madrid

 900 373 379  info@u-tad.com

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:



PROYECTO COFINANCIADO POR:

